

04. 拿捏 Git 与 GitHub

- **定位：**从零开始掌握版本控制和开源协作，让你的代码有历史、有备份、有观众。
- **核心目标：**学完本课程，你将能独立用 Git 管理自己的项目，用 GitHub 开源发布你的代码，并参与开源社区的协作。

目录

- 前言：为什么你的代码需要一个“时光机”
- 第一章：Git 是什么？——你的代码时光机
 - 1.1 没有 Git 的世界：毕业论文的地狱版本号
 - 1.2 Git 是如何实现版本管理的？
 - 1.3 Git 和 GitHub 是什么？
 - 1.4 安装 Git
 - 1.5 仓库是什么？—— `git init` 创建你的第一个仓库

前言：为什么你的代码需要一个“时光机”

Kate 在 03 学完了 Vim。她看着自己电脑上那些文件

—— `毕业论文_最终版.doc`、`毕业论文_最终版2.doc`、`毕业论文_打死也不改版.doc`——摇了摇头。她说：

“这些文件名……我以前就是这么存文档的。每一版都另存一个新文件，文件名字越来越长，最后自己也分不清哪一版才是最新的。改了一次，怕改坏了，又不敢把旧文件删掉——结果桌面上堆了十几个版本。”

“你现在知道你缺什么了吗？”

“一个能帮我记住每一版改了什么的工具？”

“对。它叫 Git。你用 Markdown 写笔记、用 VS Code 编辑、用 Vim 在服务器上改配置——但你没有 Git，你的所有文件都是‘一次性’的。改坏了就是改坏了，没有后悔药。Git 给你的文件装上时光机。”

为什么你需要 Git？

你写笔记、写教程、写代码——文件在变。今天加一段，明天删一行，后天改个错别字。没有 Git，你只能用“另存为”来保存不同版本。三个月后你回头找“昨天下午那个还没改崩的版本”，发现桌面上有 15 个文件，名字从 最终版 到 最终版7 到 打死也不改版 到 真的不改版 到 再改是狗。

Git 把你从这种泥潭里拉出来。它记录的是**修改历史**，不是文件的完整副本。你每完成一次阶段性改动，就做一次“提交”（commit）。Git 记录下这次提交的所有修改——哪个文件改了、改了哪一行、什么时候改的、谁改的。以后任何时候，你可以回顾任何一次提交，对比任意两次提交之间的差异，把文件恢复到任何一个历史状态。你不再需要“另存为”了——你只需要做一次提交，Git 就帮你记住了那一刻的文件状态。

但 Git 的价值不只在于“后悔药”。Git 还有一个更深远的意义：**让协作成为可能**。你不是一个人在写东西。你和其他人共同维护一个项目——每个人在自己的电脑上修改文件，通过 Git 把修改合并到一起，同时保留每个人改了什么、什么时候改的、为什么这样改。没有 Git，协作只能靠“发文件给我，我改完发给你”。有了 Git，全世界几百万程序员可以同时维护同一个项目——你能想象这是怎么办到的吗？答案就在这本教程里。

这本教程怎么带你学 Git?

Git 的核心操作不多

—— `init`、`add`、`commit`、`log`、`diff`、`restore`、`branch`、`merge`、`push`、`pull`。十个命令，覆盖了你 90% 的工作。但 Git 不是靠背命令学会的。它是一套**工作流**——一种组织文件修改的方式。你需要在一个真实的项目里反复练习，直到这些命令成为你的肌肉记忆。

每一个学 Git 的人都应该在真实项目上练习。这本教程的每一个“动手”环节都围绕**《守夜者之书》仓库**展开。第一章你创建仓库，第二章你第一次提交，第三章你查看历史，第四章你撤销和回滚，第五章你在分支上写教程，第六章你把它推送到 GitHub，第七章你参与开源协作，第八章你把它发布成网站，第九章你整理提交历史、发布版本标签。

到第十章结束时，你的《守夜者笔记》仓库已经推送到了 GitHub，有了完整的提交历史，甚至被发布到了 GitHub Pages。你不再是一个只会写文件的作者——你是一个用 Git 管理项目、用 GitHub 托管项目、用 Pull Request 参与开源协作的开发者。

前置知识

你不需要任何编程基础。你只需要会使用终端——在 05 第三章你已经学过基本的命令行操作，知道怎么 `cd` 切换目录、怎么输入命令、怎么看输出。你在 01 学过 Markdown，你的所有笔记和教程都是 `.md` 文件。Git 天生适合管理 Markdown 文件——纯文本格式让 Git 能精确追踪每一行改动，你在 01 里踩过的 Markdown 空格坑，Git 一眼就能帮你找出来。

怎么学?

每一章先讲概念，再动手练习。概念部分只讲你当下需要知道的——不多也不少。动手部分用《守夜者之书》仓库做主线项目，每一步都踩在前一步的基础上。

你不需要一次全懂。Git 的学习曲线不是一条直线——它像爬山，有一些陡坡。分支和合并是很多人的卡点。没关系，卡住了就回到动手环节，把命令再敲一遍。Git 不是在脑子里学会的，是在手指上学会的。

Kate 打开终端，看到光标在闪。她说：“来吧，给我的《守夜者笔记》装上时光机。”

“好。第一章，我们先认识一下 Git——你的代码时光机。”

开始阅读：第一章：Git 是什么？——你的代码时光机 

第一章：Git 是什么？——你的代码时光机

Kate 在前言里看到了那些“毕业论文_最终版.doc”的故事。她问了一个问题：

“我确实在用‘另存为’管理版本，而且效果很糟。但 Git 真的能解决这个问题吗？它到底是怎么记录我的每一次修改的？”

“好问题。这一节先让你看到没有 Git 的世界有多痛苦，然后告诉你 Git 是怎么把这个问题解决掉的。”

1.1 没有 Git 的世界：毕业论文的地狱版本号

想象你在写毕业论文。第一天，你写好了初稿，保存为 论文.doc 。

- 第二天，导师说“第一章的引言要改一下”。你改了，怕改坏，把新版本另存为 论文_修改版.doc 。
- 第三天，导师说“引用格式不对”。你改了，又另存为 论文_修改版2.doc 。
- 第四天，导师说“参考文献太少，加十篇”。你改了，又另存为 论文_修改版3.doc 。
- 第五天，导师说“还是用回修改版2的引言，但保留修改版3的参考文献”。你打开 论文_修改版2.doc ，复制引言，打开 论文_修改版3.doc ，粘贴引言，重新整理，另存为 论文_最终版.doc 。
- 第六天，导师说“这个最终版的基础还是有问题，用回修改版1，但加上修改版2的第三章”。你开始打开三个文件对照着看，眼睛在三个窗口之间来回扫，试图手动合并不同版本的优点。此时你的桌面上有：

论文.doc
论文_修改版.doc
论文_修改版2.doc
论文_修改版3.doc
论文_最终版.doc
论文_最终版2.doc
论文_打死也不改版.doc
论文_真的不改版.doc
论文_再改是狗.doc

九个文件，每个文件里都有一些“这一段是好的，那一段是废的”。你不知道哪个文件是最新的，不知道哪个文件里哪个段落是你要的，不知道“修改版2”和“最终版”之间到底差了什么。你只知道一件事：**再这样下去要疯了。**

Kate 说：“我看着我自己的桌面——就是这样的。每个版本都留着，因为不敢删。但每个版本之间差了哪几行，我完全不知道。我只能一个一个打开对比。”

“这就是没有 Git 的世界。你保存的是文件的**完整副本**——每个版本都是完整的一份，版本之间差了什么，全靠你肉眼对比。文件越多，越乱，越不敢删，越不知道哪个是最新的。”

Git 解决这个问题方式完全不同。

Git 不保存“完整副本”。它保存的是**修改记录**。

你写好了初稿，做一次提交——Git 记录下整个文件的当前状态，给它拍了一张“快照”，标上“这是第一版”。

你改了引言，做一次提交——Git 记录下**你改了哪些行**，以及“这是第二版，基于第一版改的”。

你改了引用格式，做一次提交——Git 记录下**你改了哪些行**，以及“这是第三版，基于第二版改的”。

你加了十篇参考文献，做一次提交——Git 记录下**你改了哪些行**，以及“这是第四版，基于第三版改的”。

现在你可以做这些事：

- **查看任何一个历史版本**：一条命令，回到“第一版”的状态——初稿，什么都没有改。
- **对比任意两个版本**：一条命令，显示“第二版和第三版之间到底差了什么”——哪一行被改了、改成什么了。
- **回到任何一个版本**：你觉得第四版写砸了，一条命令，文件恢复到第三版。
- **合并不同版本的修改**：导师说“用第三版的引言，但要保留第四版的参考文献”——Git 帮你自动合并，你只需要处理冲突的部分。

你的桌面上不再有九个文件了。你只有一个文件——`论文.md`（用 Markdown 写，你已经在 01 学过了）。和它一起的，是一长串提交记录，每条记录都告诉你“这次改了什么、为什么改、什么时候改的”。这不是九个文件——这是一条时间线。你可以在时间线上自由穿梭。

Kate 说：“所以 Git 不是‘自动保存’——自动保存是保存最新版。Git 是保存**每一个版本**，并且记住版本之间的关系。我把论文比作一条河，Git 就是河上的桥——我可以在任何一个桥墩上停下来，看看那时候河水的样子。”

1.2 Git 是如何实现版本管理的？

Kate 在上一节看到了 Git 能做什么。她靠在椅背上，想了一会儿，然后说：

“我知道了 Git 能回到任何历史版本、能对比不同版本的差异。但我有点好奇——它到底是怎么做到的？它是不是把我的每个版本都完整存了一份？那我的硬盘不是早就爆了？”

“这个问题问得很有档次。Git 的实现方式确实不是‘每个版本存一份完整副本’。它用了一套很巧妙的设计。我不给你讲代码，只讲思想——这套思想，是你理解 Git 所有行为的关键。”

核心思想：快照，而不是差异

大多数版本管理系统的思路是：**记录文件每次改了什么**。就像记账：

版本1：有 A、B、C 三个文件
版本2：A 文件第 3 行加了 xxx，B 文件删了
版本3：C 文件改了 yyy

Git 不这么干。Git 的思路是：**每次提交，直接给整个项目拍一张“全家福”**。

你没看错，是**整个项目**。所有文件的当前状态，咔嚓一下，全存下来。

这时候你肯定想：这不得把硬盘撑爆？

这就是 Git 最精妙的地方。

文件的“身份证”：内容指纹

Git 不给文件起名字——它给每个文件的**内容**计算一串独特的字符。这串字符叫“哈希值”。它有两个特性：

1. **唯一性**：同样的内容，永远算出同一串字符。内容哪怕改了一个标点，算出来的字符就完全不一样了。

2. **不可逆**：从这串字符反推不出文件内容。

这串字符，就成了文件内容的“指纹”。指纹一样，内容就一样。指纹不一样，内容一定改过了。

怎么存？——“只要指纹没变，就不用再存一遍”

当你提交一个新版本时，Git 会遍历项目里所有文件：

- **对于内容没变的文件**：它的指纹跟上一个版本一样。Git 说：“这个文件我仓库里已经有了，不用再存了，记个引用就行。”
- **对于内容变了的文件**：计算新的指纹，把新内容存进去。

所以，你看起来 Git 给每个版本都拍了“整个项目的快照”，实际上，**那些没变动的文件，只是在新快照里记了个引用，并没有占用新的存储空间。**

这就像你每年拍一张全家福。照片上的人，今年和去年长得一样，就不用重新“制造”这个人——在照片上标注“还是去年那个人”就行。只有变了的人，才需要更新。

版本本身长什么样？——一条链子

每次提交，Git 都会创建一个“提交记录”。这个记录里存着四样东西：

1. 这个版本的项目快照——项目里所有文件的指纹
2. 作者和时间——谁在什么时候提交的
3. 提交说明——你写的 commit message
4. **指向上一个版本的指针**——关键！它记住了“我是基于哪一个版本改出来的”

这个“指向上一个版本”的设计，让所有版本串联成了一条链子：

```
A → B → C → D
↑   ↑   ↑   ↑
v1  v2  v3  v4
```

每个节点都是那个时刻完整项目的快照。你想回到 B 版本？Git 只需要找到 B 这个提交记录，然后根据它记录的文件指纹，把所有文件还原出来。

这就是**时空穿越**的本质。不是“撤销”了改动，而是重新“播放”了那个瞬间的一切。

分支：只是一个会移动的指针

你之前听过“分支”这个词吗？在 Git 里，分支简单得让人吃惊。

Git 有一个特殊指针叫 `HEAD`，它指向你当前在哪个位置。而一个分支名（比如 `main`、`dev`），本质上就是一个**指向某个提交记录的指针**。

当你新建一个分支并提交时：

- Git 创建一个新的提交记录，它的“上一个版本”指向你当时所在的那个版本
- 然后把分支指针移到这个新提交上

两个分支共享着相同的“历史基础”，只是因为指针指向了不同的提交，就可以各自往下生长。分支不是一个副本——它只是一个标签，贴在某一个版本上。

合并为什么通常很轻松？

因为 Git 的合并，不是比对你的文件和我的文件“哪里不一样”。而是：

找到两个分支的共同起点，然后看：

“从起点到我，改了这些文件。”

“从起点到你，改了这些文件。”

——如果我们改的是不同文件，或者同一个文件的不同位置，Git 直接自动合并。只有我们改了同一个文件的同一行，Git 才会说：“我搞不定了，你来吧。”（这就是“冲突”。）

总结：Git 的版本管理，是一套“基于内容指纹的智能档案库”

它不是给文件编号，不是存差异，而是：

用文件内容的指纹作为索引，构建了一个“只要没变，就不用重存”的智能档案库。然后用链子组织版本，用指针管理分支。

所以，Git 的仓库为什么这么“瘦”？因为它**几乎不存重复的东西**。为什么可以随便建分支？因为**分支只是一个指针，几乎是零成本**。为什么可以飞速穿梭历史？因为它还原一个版本，就是把那个时刻的“文件指纹”指过去——瞬间完成。

你所有的代码、回滚、分支、合并，背后都是这套优雅的、基于指纹的“时空档案系统”在默默运转。

从这个角度看，Git 与其说是一个版本管理工具，不如说是一座用指纹建造的“时间博物馆”。每一个 commit，都是那个时刻你项目的完美复制品，静静地躺在馆里，等待你随时穿越回去。

Kate 听完，沉默了几秒。然后她说：“所以 Git 不存重复的东西——它只存变化的文件。没变的文件，它就记一句‘和上个版本一样’。这就像一个聪明的图书管理员，不会把同一本书抄十遍，只是在目录里标注‘这本书在 A 书架第三层’。”

“对。而且这个图书管理员有一双极其敏锐的眼睛——文件内容哪怕改了一个标点，它都能立刻认出来。凭的就是那个内容指纹。指纹一样就是一样，不一样就是不一样，没有模棱两可。”

“……那这个图书管理员住在我电脑上吗？我怎么才能用到它？”

“它确实住在你电脑上——但它不是图形界面，是一个命令行工具。而且它有一个孪生兄弟叫 GitHub，专门负责把你的时间博物馆放到网上，让全世界都能看到。下一节，我们先把 Git 和 GitHub 这两个东西分清楚，然后把它们请到你的电脑上。”

下一节，我们要学习 **Git 和 GitHub 是什么**——一个是你电脑上的版本管理工具，一个是托管仓库的网站。很多人把它俩混为一谈，但它们是完全不同的两样东西。✍

1.3 Git 和 GitHub 是什么？

“我大概明白了——Git 是一个聪明的图书管理员，用指纹识别文件变化，给项目建了一座时间博物馆。但我还有一个问题：Git 和 GitHub 是一回事吗？我总听人把这两个词连在一起说，好像它们是同一个东西。”

“好问题。这一节把这两个名字拆开，顺便说说 Git 是从哪来的。”

1.3.1 什么是版本管理？

在讲 Git 和 GitHub 之前，先把“版本管理”这个词说清楚。你在 1.1 节已经感受过没有版本管理的痛苦——九个毕业论文文件，不知道哪个是最新的。版本管理，就是**帮你管理文件每一次改动的工具**。

你写的每一篇笔记、每一章教程、每一段代码，都在不停变化。今天加一段，明天删一行，后天改个错别字。版本管理工具负责记录这些变化——什么时候改的、改了哪里、改成什么了。让你随时可以回头查看“昨天下午那个还没改崩的版本”，对比“这一版和上一版差了什么”，甚至在改坏了的时候一键回到历史状态。

于是你想，如果有一个这样的软件，不但能自动帮我记录每次文件的改动，还可以让导师协作编辑，这样就不用自己管理一堆类似的文件了，也不需要把文件传来传去。如果想查看某次改动，只需要在软件里瞄一眼就可以，岂不是很方便？

这个理想软件用起来就应该像这个样子，能记录每次文件的改动：

版本	文件	用户	说明	日期
1	论文.doc	Kate	写好了初稿	X 月 X 日
2	论文.doc	导师	改了第一章的引言要	X 月 X 日
3	论文.doc	Kate	更改引用格式	X 月 X 日
4	论文.doc	Kate	参考文献太少，加十篇	X 月 X 日
5	论文.doc	Kate	用回版本 2 的引言，但保留版本 3 的参考文献	X 月 X 日
6	论文.doc	导师	用回版本 1，但加上版本 2 的第三章	X 月 X 日

这样，你就结束了手动管理多个“版本”的史前时代，真正进入到了版本控制。

Git 是目前最流行的版本管理工具——不是因为它是第一个，而是因为它最好用、最快、最灵活。

1.3.2 Git 的起源与历史

Git 诞生于 2005 年，作者是 **Linus Torvalds**。

这个名字你将会在以后读到他的很多故事。1991 年，21 岁的芬兰大学生 Linus 写了一个“业余项目”叫 Linux，发布到网上让所有人免费使用。

——三十年后，Linux 运行在全世界 90% 以上的服务器上，整个互联网的“地基”就是它。

Linus 虽然创建了 Linux，但 Linux 的壮大是靠全世界热心的志愿者参与的，来自全球几百家公司的几千名开发者为 Linux 编写代码，那 Linux 的代码是如何管理的呢？

在 2002 年以前，世界各地的志愿者把源代码文件通过 diff 的方式发给 Linus，然后由 Linus 本人通过手工方式合并代码！

管理这么大规模的项目，肯定需要一个版本管理工具。

所以到了 2002 年，Linux 系统已经发展了十年了，代码库之大让 Linus 很难继续通过手工方式管理了，社区的弟兄们也对这种方式表达了强烈不满，于是 Linus 选择了一个商业的版本控制系统 BitKeeper，BitKeeper 的东家 BitMover 公司出于人道主义精神，授权 Linux 社区免费使用这个版本控制系统。

接下来发生的事情嘛……

“到了 2005 年，开发 BitKeeper 的商业公司同 Linux 内核开源社区的合作关系结束，他们收回了 Linux 内核社区免费使用 BitKeeper 的权力。”

——Git 官方文档

但根据野史记载实际却是社区牛人聚集，不免沾染了一些梁山好汉的江湖习气。Andrew 试图破解 BitKeeper 的协议（这么干的其实也不只他一个），被 BitMover 公司发现了，于是 BitMover 公司怒了，要收回 Linux 社区的免费使用权。

Linus 可以向 BitMover 公司道个歉，保证以后严格管教弟兄们，嗯，这是不可能的。实际情况是这样的：

Linus 用了**两周**时间，用 C 语言写完了 Git 的核心设计。他的目标很明确：**快、分布式、安全**。

- **快**：Git 的哈希快照设计让它在大规模项目上依然飞快。几千名开发者同时提交，Git 处理得过来。
- **分布式**：每个开发者的电脑上都有一份完整的仓库。你可以离线工作，不需要时刻连着服务器。
- **安全**：用内容指纹（哈希值）来验证文件是否被篡改。文件如果被改过，Git 一定能发现。

“Git”这个名字是 Linus 起的。在英式俚语里，“git”是“蠢货”“讨厌的人”的意思。Linus 自嘲说：“我自私，我把我所有的项目都用自己的名字命名。Linux 是 Linus 的，Git 是 git 的。”（这个“git”指的是他自

己——他觉得自己就是个“讨厌的蠢货”。) 这种自嘲式命名，你以后在 Geek（极客）圈会经常见到——黑客喜欢自嘲。

Kate 说：“Linus 用两周写了一个全世界最流行的版本管理工具。而且他还用‘蠢货’给它命名。我在 01 里学 Markdown 的时候，也是冲着‘用键盘排版’去的。这些工具的背后，全是这种‘不高兴就自己造一个’的人。”

“对。这就是极客精神——Linus 曾说过：‘你不需要在一开始就做出完美的产品。你只需要做出一个让其他人觉得可以用、然后一起把它变得更好的东西。’Git 也是这么来的。他为了自己的需要写了第一版，然后全世界的开发者一起把它变成了今天这个样子。”

“历史总是这么巧合，如果当年 BitMover 公司没有收回版权……”

“——那么我们将不会看到 Git。”

1.3.3 什么是 GitHub?

Git 和 GitHub 不是一回事。很多人把这两个词混着用，但它们是完全不同的东西。

Git 是一个工具——你安装在自己电脑上的一个程序。它负责管理你本地文件的版本历史。你用 Git 做提交、查看历史、创建分支、回滚版本——所有操作都在你自己的电脑上完成，不需要互联网连接。

GitHub 是一个网站——一个托管 Git 仓库的在线平台。你在自己电脑上用 Git 管理好项目之后，把整个仓库“推送”（push）到 GitHub 上。GitHub 帮你把仓库托管在云端——你的代码有了一份在线备份，别人也能看到、下载、甚至提交修改。

打个比方：Git 是你的**相机**——拍照、拍视频、存储照片。GitHub 是你的**云相册**——你把照片上传上去，别人能看，你也能随时下载回来。相机和云相册是两个不同的东西——没有相机你拍不了照片，但没有云相册你也能拍照，只是照片只能自己看。

维度	Git	GitHub
是什么	版本管理工具	代码托管网站
装在哪	你自己的电脑上	互联网上的远程服务器
干什么	记录文件修改历史	存储 Git 仓库，提供协作功能
需要联网吗	不需要	必须联网
其他同类	有替代品（Mercurial 等）	同类网站有 GitLab、Gitee 等

GitHub 不只有托管仓库。 它还提供了很多 Git 之外的功能：

Issues（问题追踪）

你在 GitHub 仓库里看到了一个问题？点进 Issues 标签，新建一个 Issue，描述你遇到的问题——比如“第三章的链接跳转错了”“这个安装步骤在 Windows 11 上跑不通”。仓库的维护者会看到你的 Issue，回复你、修复问题、关闭 Issue。你在《守夜者之书》的 README 里写过“催更请至 Issues”，指的就是这个功能。

Pull Request (协作的核心)

你看到了一个开源项目，想贡献一点自己的修改——修一个 bug、加一个新功能、改一个错别字。你不是仓库的主人，不能直接改它的代码。于是你 Fork 一份到自己的账号下（等于把仓库复制了一份到你的 GitHub），在自己那份里修改，然后提交一个 Pull Request——告诉原仓库的主人：“我做了一些修改，你要不要看看？如果合适，请把我的修改合并进来。”原仓库的主人看到你的请求，审查你的修改，选择接受或拒绝。Pull Request 是开源协作的枢纽——全球所有开源项目的贡献者，都是通过它和项目维护者沟通的。

Actions (自动化)

你可以设置一个工作流——比如每次有人 push 代码，GitHub 自动运行测试、检查代码风格、部署到服务器。Actions 让你把重复性工作交给 GitHub 自动执行。你以后把《守夜者之书》仓库配置成“每次提交自动生成 PDF 版本”，用的就是 Actions。

Pages (免费静态网站托管)

你的仓库可以直接变成一个网站。你把 Markdown 文件转换成 HTML，放进一个叫 `gh-pages` 的分支里，GitHub 自动帮你发布成一个静态网站。你的《守夜者笔记》以后就可以用 Pages 发布成在线教程——读者输入一个网址就能阅读，不需要下载仓库。

这些功能让 GitHub 从“代码仓库”变成了**程序员的社交网络**。你在上面关注别人、给别人的项目点赞（Star）、Fork 别人的代码、参与讨论、贡献代码。你的 GitHub 主页，就是你的程序员名片。

Kate 说：“所以 Git 是我电脑里的相机，GitHub 是云相册。我在 01 里给仓库配的徽章、写的 README——那是云相册里的展示页。但我还没学会怎么用相机拍照、怎么把照片传到云相册。”

“对。但在拍照之前，你手里还没有相机。下一节，我们把相机装上——安装 Git。”

下一节，我们要学习 **安装 Git**——在你的系统上安装 Git，配置你的用户名和邮箱，为第一次提交做准备。🔪

1.4 安装 Git

Kate 在上一节搞清楚了 Git 和 GitHub 的区别。她说：“Git 是相机，GitHub 是云相册——这个比喻我记住了。但我电脑上现在没有相机。我先得把 Git 装上。”

“对。装 Git 很快。你只需要根据你的操作系统选一种方式。”

1.4.1 Windows 安装 Git

打开 PowerShell 或 CMD，输入：

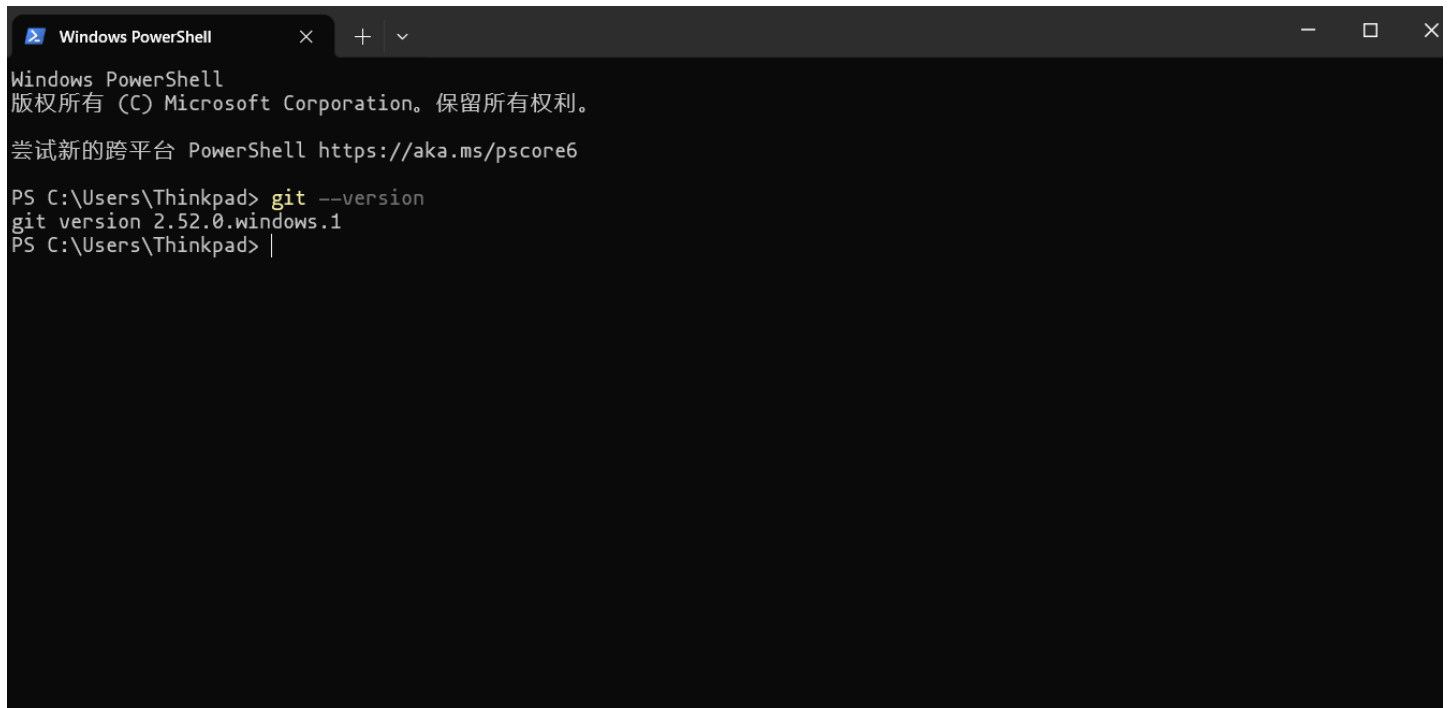
```
winget install Git.Git
```

`winget` 是 Windows 自带的包管理工具。这行命令会从微软的软件仓库下载并安装 Git。

安装完成后，**关闭当前的终端窗口，重新打开一个新窗口**（这一步很重要——不重新打开，终端不会识别新安装的 Git）。然后输入：

```
git --version
```

如果输出版本号（比如 `git version 2.47.x`），安装成功。

A screenshot of a Windows PowerShell terminal window. The title bar says "Windows PowerShell" with standard window controls. The terminal text shows the copyright notice for Microsoft Corporation, a link to the PowerShell website, and then the command `git --version` being executed. The output is `git version 2.52.0.windows.1`. The prompt is `PS C:\Users\Thinkpad>`.

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Users\Thinkpad> git --version
git version 2.52.0.windows.1
PS C:\Users\Thinkpad> |
```

1.4.2 macOS 安装 Git

打开终端，先安装 Homebrew（如果你在 05 第三章已经装过了，跳过这一步）：

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

然后用 Homebrew 安装 Git：

```
brew install git
```

验证：

```
git --version
```

1.4.3 Linux 安装 Git

在终端里输入：

```
sudo apt install git -y
```

验证：

```
git --version
```

1.4.4 配置你的身份

Git 装好之后，你需要告诉它“你是谁”。每一次提交都会附带你的身份信息——用户名和邮箱。这些信息会出现在提交记录里，告诉别人（和未来的你自己）这次提交是谁做的。

在终端里输入：

```
git config --global user.name "你的名字"  
git config --global user.email "你的邮箱"
```

`--global` 表示这个配置对你的电脑上所有 Git 仓库生效，不只是当前这一个。你只需要配置一次。

你可以用你 GitHub 账号的用户名和邮箱——这样以后推送到 GitHub 时，提交记录会自动关联到你的 GitHub 账号，你的 GitHub 主页上就会显示你的贡献记录（那些绿色的小方块）。

验证配置是否成功：

```
git config --list
```

会列出你所有的 Git 配置，包括刚刚设置的用户名和邮箱。

Kate 输入了 `git config --global user.name "Kate"` 和 `git config --global user.email "kate@example.com"`，然后 `git config --list` 查看了配置。她说：“所以 Git 现在知道我是谁了。以后我做的每一次提交，都会标上我的名字和邮箱。”

“对。而且这些信息是永久记录在提交历史里的——不管过了多久，别人翻到你的提交记录，都知道那一次是你做的。所以用户名和邮箱要填真实的、你愿意公开的。”

“……我现在填的是 Kate。我可以用真名吗？”

“可以，也可以一直用 Kate。取决于你想不想让未来的同事和面试官知道这个仓库是你做的。很多人用真实姓名，也有很多人用网名。这是你的选择。”

修改用户名和邮箱

如果配置的时候手滑拼错了，或者以后想换一个名字（比如你之前用真名，现在想换成网名），不需要重新安装 Git——直接重新运行配置命令，新值会覆盖旧值：

```
git config --global user.name "新的名字"
git config --global user.email "新的邮箱"
```

改完之后，用 `git config --list` 验证。以后的新提交会使用新的名字和邮箱。注意：**之前已经提交过的历史记录不会改变**——旧提交上仍然显示旧名字。Git 只在新提交里使用新配置。如果你想让旧提交也换成新名字，需要改写历史——那是第九章 `git rebase` 的内容，现在不用管。

查看单独某一项配置：

```
git config --global user.name
git config --global user.email
```

不需要看整个列表，只查你想看的那一项。

把默认分支名设为 main

Git 的默认分支名曾经是 `master`。2020 年，全球开发者社区发起了一场行动，把它改成了更中性的 `main`——GitHub 随后也采用了 `main` 作为新仓库的默认分支名。但有些版本的 Git 在你电脑本地创建新仓库时，仍然默认使用 `master`。

为了统一，你可以在 Git 配置里设置默认分支名为 `main`：

```
git config --global init.defaultBranch main
```

这条命令的意思是：以后每次用 `git init` 创建新仓库，默认分支名都是 `main`，而不是 `master`。你在 5.3 节学分支策略时用到的“主线”，指的就是这个 `main`。

验证配置是否生效：

```
git config --global init.defaultBranch
```

看到 `init.defaultbranch=main` 就说明配置成功了。

Kate 关掉了终端，又打开了一个新窗口，输入 `git --version` ——版本号正常显示。她说：“相机装好了，电池也装上了。接下来怎么拍照？”

“下一节，我们创建你的第一个 Git 仓库。把你的《守夜者笔记》文件夹变成一个有历史记录的项目。”*

1.5 仓库是什么？—— `git init` 创建你的第一个仓库

Kate 在上一节配好了名字、邮箱、默认分支名。她打开终端，盯着一行空白的命令行，说：

“相机装好了。现在怎么拍照？我是不是要先把照片存在一个相册里？”

“对。那个相册就叫仓库。我们先用 `git init` 创建它。”

1.5.1 仓库是什么？

仓库 (repository, 缩写 repo) 是 Git 用来存放项目的一个**文件夹**。这个文件夹和普通文件夹的区别只有一点：它里面多了一个隐藏的 `.git` 目录。这个 `.git` 目录是 Git 的“监控室”——你所有的提交记录、分支、配置、文件指纹全存在这里。有了 `.git`，你就能用 Git 追踪这个文件夹里所有文件的变动。

创建仓库的方式就是进入项目文件夹，输入 `git init`。这条命令会在当前文件夹里建好 `.git` 目录，把这个普通文件夹变成 Git 仓库。

Kate 说：“听起来好简单——就一个隐藏文件夹。那我之前桌面上那一大堆毕业论文文件夹，其实只要在其中一个里面输入 `git init`，它就能变成仓库了？”

“对。而且这个 `.git` 目录很轻量。你在 1.2 节学过 Git 用文件指纹去重，所以 `.git` 里不会存一堆重复副本。”

1.5.2 动手：创建你的第一个仓库

第一步：创建一个项目文件夹

在你的电脑上找一个你找得到的地方（比如桌面或 `D:\Dev`），创建一个文件夹。我们给它起名叫 `my-first-repo`：

```
mkdir my-first-repo
```

第二步：进入这个文件夹

```
cd my-first-repo
```

第三步：初始化仓库

```
git init
```

终端会显示类似这样的输出：

```
Initialized empty Git repository in /path/to/my-first-repo/.git/
```

这句话的意思：“在这个文件夹里创建了一个空的 Git 仓库。”

第四步：验证 `.git` 目录已经创建

输入：

```
ls -a
```

`-a` 参数让 `ls` 显示隐藏文件。你会看到：

```
.  ..  .git
```

那个 `.git` 目录就是 Git 的监控室。你的文件夹现在是一个 Git 仓库了。

Kate 在终端的输出里看到 `.git`，说：“这就完了？就三行命令，我的文件夹就变成仓库了？我还以为要填一堆表格、做一堆配置。”

“就这三行。Git 的很多操作都是这样——命令很短，但每一行都在做一件具体的事。`mkdir` 建文件夹，`cd` 进去，`git init` 变成仓库。”

1.5.3 仓库里什么还没有？

`git init` 创建的仓库是**空的**。它知道你要开始追踪文件了，但还没有追踪任何文件。你的项目文件夹里目前也什么都没有——没有 README，没有教程，没有代码。接下来的章节，你会往这个文件夹里放文件，用 `git add` 把它们加入追踪，用 `git commit` 把它们的当前状态拍成快照。

Kate 用 `ls -a` 确认了 `.git` 目录存在，然后问：“那我现在这个仓库里有历史吗？”

“还没有。一个空仓库，连一次提交都没有。它像一个刚开张的博物馆——建筑建好了，但展厅里一件展品都没有。下一章，我们往里面放第一件展品，然后给它拍第一张快照。”

“.....拍照之前，我得先知道怎么拍。”

“这就是第二章的内容。`git add` 把文件放进相机的取景框，`git commit` 按下快门。这两条命令，是你以后每一天都会用到的核心。你的第一张照片，就是你的代码的第一个里程碑。”

下一章，我们要学习**第二章：第一次提交——你的代码里程碑**。你将把文件加入 Git 的追踪，创建你的第一次提交，在自己的仓库里留下第一个历史节点。🔪